Software Evolution

Software evolution refers to the continuous process of developing, modifying, and adapting software products to meet changing requirements, technologies, and environments throughout their lifecycle. Unlike physical systems, software is not consumed through use, it ages when it fails to adapt to new demands. In real-world terms, after a software product is delivered, it doesn't remain static. New user needs emerge, bugs are discovered, technologies evolve, and regulations change. To remain useful, software must therefore evolve. "Software evolution is the process of changing a software system to keep it useful in a changing environment." Thus, software evolution ensures relevance, usability, and competitiveness of a software product over time. Software evolution is not optional; it is an inevitable reality in the life of every software system.

Software Maintenance

Software maintenance is the process of modifying and updating software after its initial delivery to correct faults, improve performance, or adapt it to a changed environment. It represents the longest and costliest phase of the software lifecycle — often consuming 60–80% of total software costs. "Software maintenance is the modification of a software product after delivery to correct faults, to improve performance, or to adapt the product to a modified environment." Through effective maintenance, developers keep systems running.

Types of Software Maintenance

1. Corrective Maintenance: Fixing defects discovered during usage. Example: Removing bugs that cause the application to crash.
2. Adaptive Maintenance: Modifying software to run in a new or changed environment. Example: Updating an application to work with a new version of Windows or a new database system.
3. Perfective Maintenance: Enhancing system performance or maintainability, even if no faults exist. Example: Improving the interface for better user experience.
4. Preventive Maintenance: Making changes to prevent future problems. Example: Refactoring code to reduce technical debt and future failures.

Importance of Maintenance

1. Ensures continuous usability
2. Fixes emerging faults and vulnerabilities
3. Adapts the system to new technologies
4. Improves user satisfaction and performance
5. Extends software lifespan

Software Re-engineering

Software re-engineering is the process of analyzing and modifying existing software systems to improve their structure, functionality, or performance without altering their external behaviour. It is essentially "revitalizing old software" instead of building a new one from scratch. It means "modernizing old software to meet current standards and technologies."

Objectives Software Re-engineering

1. Improve system quality and performance
2. Reduce maintenance complexity
3. Extend system lifespan
4. Adapt legacy systems to new platforms or technologies

Steps in Re-engineering Process

1. Inventory Analysis: Identifying which software components are worth re-engineering.
2. Reverse Engineering: Extracting design and specifications from existing code to understand the system.
3. Code Restructuring: Cleaning up and reorganizing poorly written code.
4. Data Restructuring: Reorganizing data models, database schemas, or file formats.
5. Forward Engineering: Rebuilding the system with improved design and modern technologies.

A Nigerian university still using a 2005-built student record system may re-engineer it to: Use web-based technologies, Integrate with mobile devices, Support modern databases like PostgreSQL or cloud storage

Legacy Systems

A legacy system is an old software or hardware system that continues to be used because it still performs essential functions, even though it may be outdated or difficult to maintain. These systems often form the backbone of organizations (e.g., in universities, banks, or government agencies), but they may rely on obsolete technologies.

Characteristics of Legacy Systems

1. Built with old technologies or programming languages (e.g., COBOL, FORTRAN)
2. Poor or missing documentation
3. Hard to modify or extend
4. Dependence on old hardware or operating systems
5. High maintenance cost but critical for operations

Management of Legacy Systems

Encapsulation: Wrap the old system with an interface to interact with new systems.

Migration: Gradually move data and functionality to modern platforms.

Replacement: Completely rebuild the system with modern tools.

Re-engineering: Redesign internal components without altering external behavior.

Software Reuse

Software reuse is the practice of using existing software components, designs, or knowledge in new applications. It helps to save time, reduce cost, and improve reliability by leveraging proven solutions. Simple Meaning: “Don’t reinvent the wheel — use what already works.”

Forms of Software Reuse

1. Code Reuse: Using existing code libraries or modules in new projects (e.g., open-source libraries).
2. Design Reuse: Reusing design patterns, templates, or architecture frameworks.
3. Component Reuse: Integrating prebuilt components or APIs into new systems (e.g., payment gateway APIs).
4. Test Reuse: Reusing test cases and scripts for similar software systems.
5. Documentation Reuse: Reapplying templates, reports, or requirement formats in future projects.

Benefits of Software Reuse

1. Reduced development cost and time
2. Increased reliability (tested components are reused)
3. Improved productivity
4. Easier maintenance
5. Encourages standardization

Challenges of Software Reuse

1. Compatibility issues
2. Licensing or intellectual property concerns
3. Poor documentation of reusable components
4. Difficulty adapting reused components to new contexts